

CAMBRIDGE INSTITUTE OF TECHNOLOGY

K.R. PURAM, BANGALORE-560 036

Dept. of CSE (Data Science)



Principle of Data Science

(CD254)

IV Semester

LAB MANUAL

Prepared by

Ms.Divya Sugathan

Contents

1. Install & Explore Packages	3
2. Implementation and Manipulation of Arrays Using NumPy	12
3. Working with Pandas DataFrames for Data Analysis	15

Program :1

Install & Explore Packages

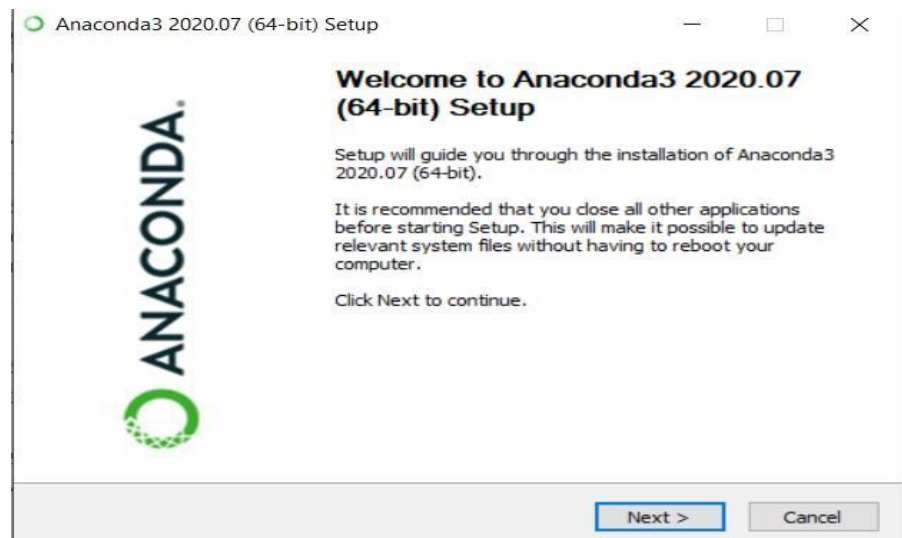
Title: Download, Install, and Explore the Features of NumPy, SciPy, Jupyter, Statsmodels, and Pandas Packages

Problem Description: This task focuses on downloading, installing, and exploring the core features of essential Python libraries used for scientific computing, data analysis, and statistical modeling. The objective is to understand the purpose and functionality of NumPy, SciPy, Jupyter Notebook, Statsmodels, and Pandas in data science and numerical analysis applications.

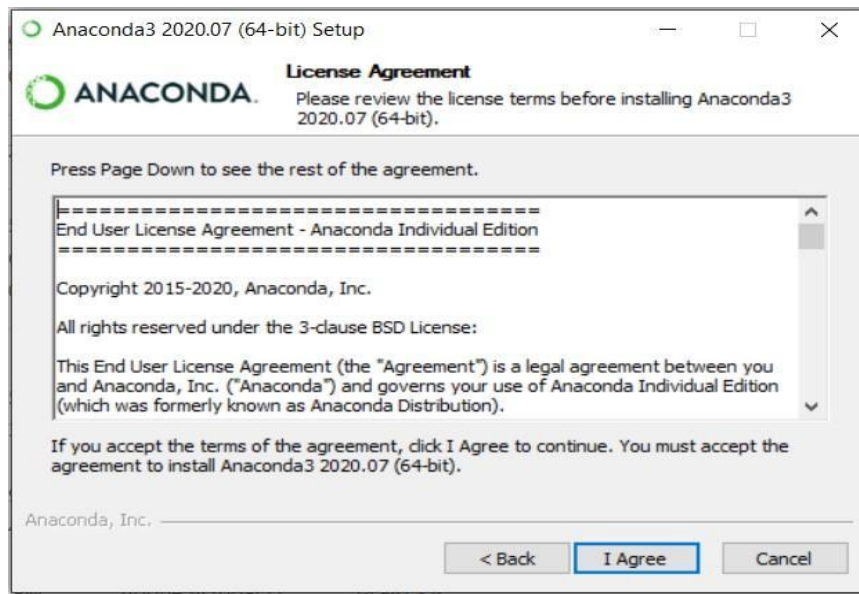
Procedure

Anaconda Installation for windows:

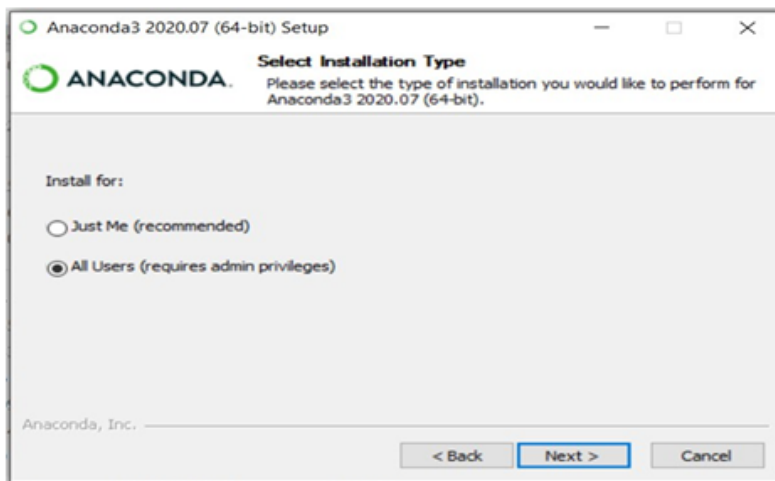
1. Download **Anaconda** according to your **system requirements or operating system**.
2. After the download is complete, **double-click the installer file to launch the setup**.
3. Then **click “Next”** to proceed with the installation process.



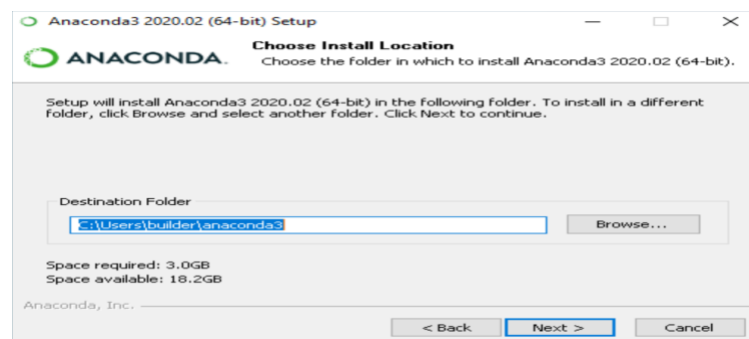
4. Read the licensing terms and click “I Agree”.



5. Select an install for “All Users” and click Next.



6. Select a destination folder to install Anaconda and click the Next button.

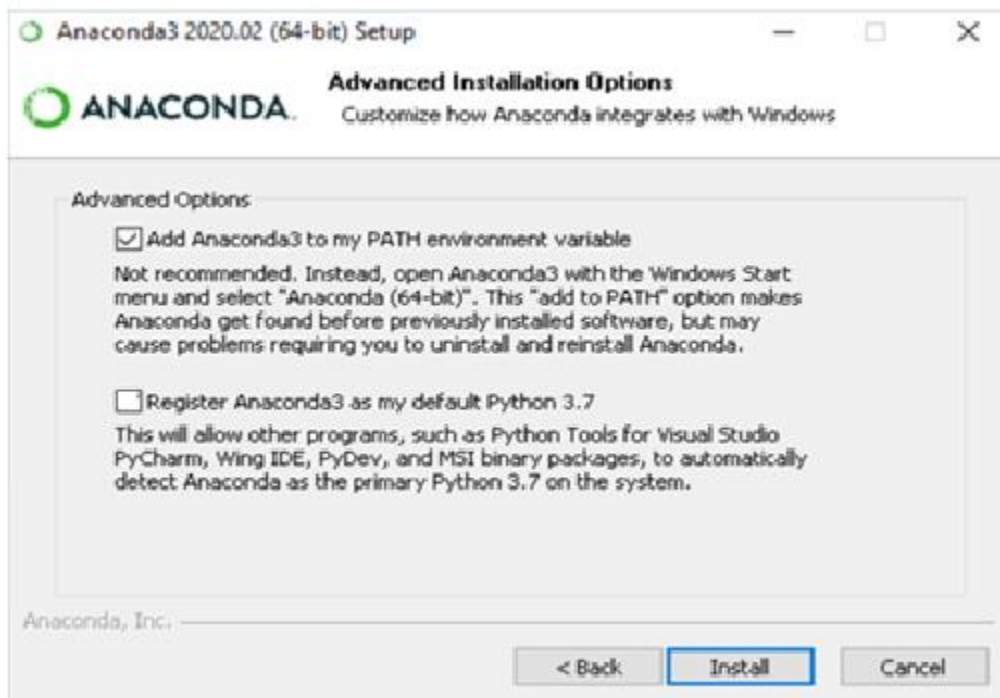


7. Choose whether to add Anaconda to your PATH environment variable. We recommend not

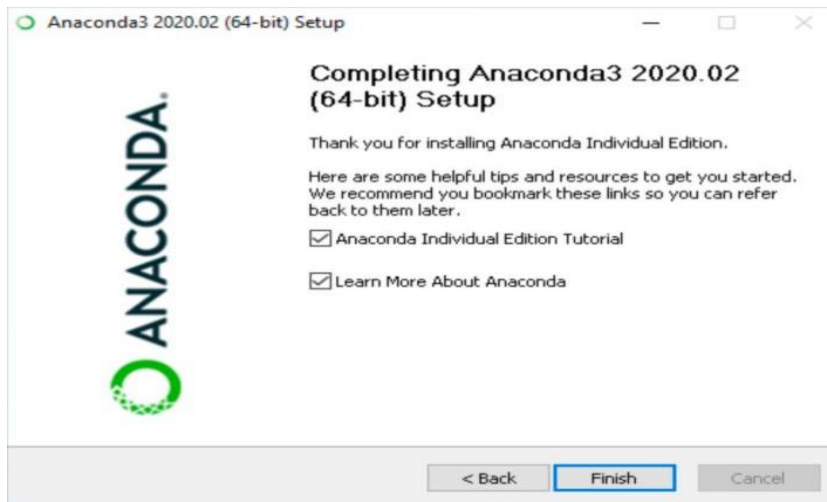
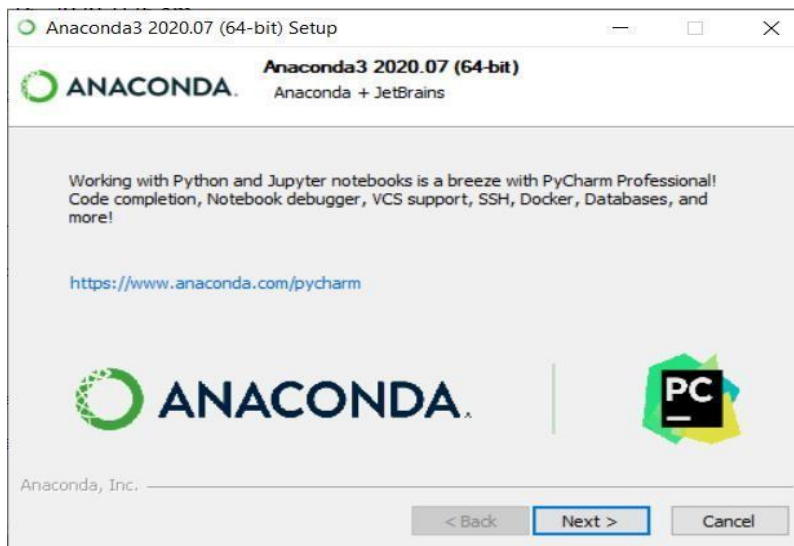
adding Anaconda to the PATH environment variable, since this can interfere with other software. Instead, use Anaconda software by opening Anaconda Navigator or the Anaconda Prompt from the Start Menu

NOTE: Choose whether to register Anaconda as your default Python. Unless you plan on installing and running multiple versions of Anaconda or multiple versions of Python, accept the default and leave this box checked.

8. Click the Install button. If you want to watch the packages Anaconda is installing, click Show Details



9. Click the Next button.



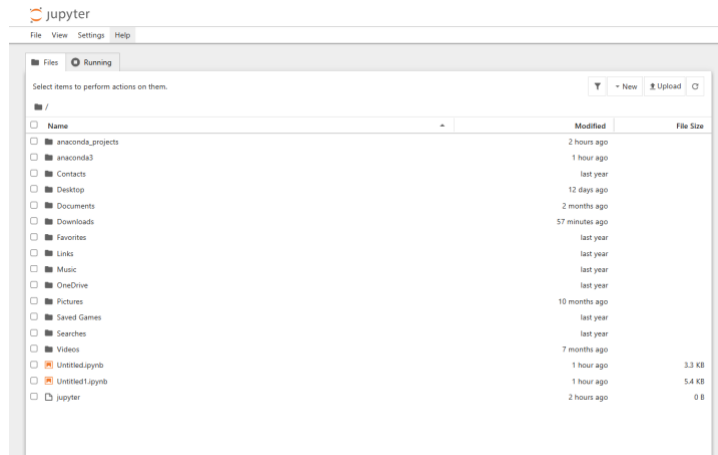
10. And then click the Finish button.

11. After a successful installation you will see the “Thanks for installing Anaconda” dialog box.

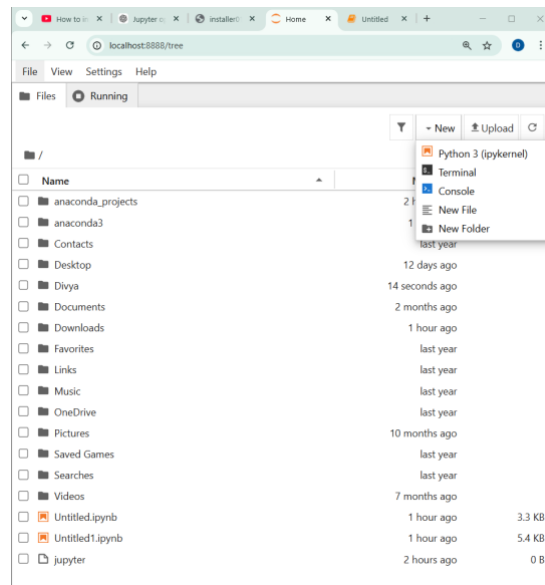
12. Go to start click on “Anaconda Prompt”.

13. Type jupyter notebook and press enter.

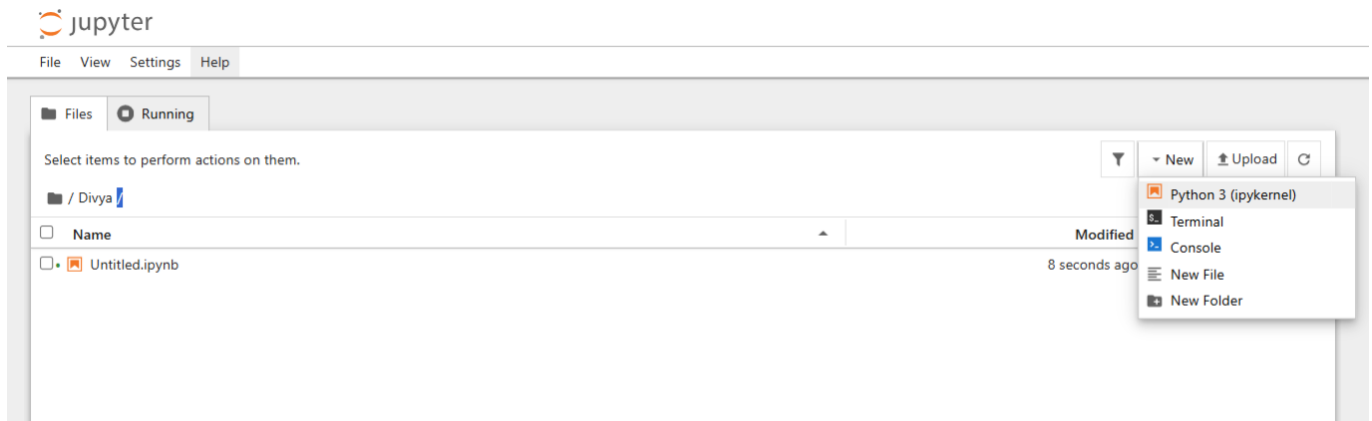
14. A web page (<http://localhost:8888>) opens in default browser.



15. Click New → Folder. Rename it by ticking the box → Rename



16. To create a new Notebook, click New (top-right corner) and select Python 3 (ipykernel)



17. Write the code within the cell .

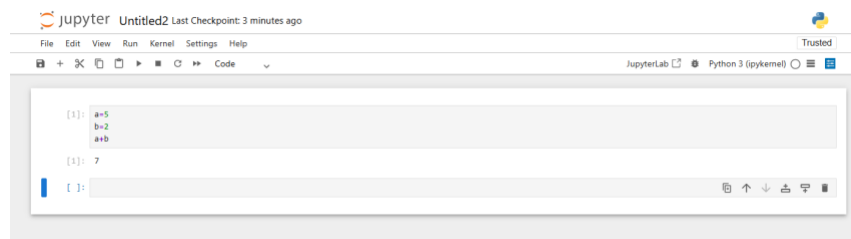
18. Verify the installed versions of essential Python libraries (**Pandas, NumPy, Matplotlib**) used for data analysis and visualization.

```
# Import the required libraries
import pandas as pd          # For data analysis
import numpy as np           # For numerical computing
import matplotlib            # For data visualization

# Print the installed version of Pandas
print("Pandas version:", pd.__version__)

# Print the installed version of NumPy
print("NumPy version:", np.__version__)

# Print the installed version of Matplotlib
print("Matplotlib version:", matplotlib.__version__)
```



RESULTS:

- Anaconda and Jupyter Notebook were successfully installed on the system.
- A new Jupyter Notebook was created, and Python 3 (ipykernel) was selected as the kernel.
- Essential Python libraries were imported without errors: Pandas, NumPy, and Matplotlib.
- The installed versions of the libraries were successfully verified and printed:

LEARNING OUTCOMES:

- Install and configure Python packages and tools such as Anaconda, Jupyter Notebook, and Spyder on a Windows system.
- Launch and navigate Jupyter Notebook to create interactive Python notebooks for coding and visualization.
- Understand the purpose and functionalities of essential Python libraries used in data science and numerical computing, including NumPy, SciPy, Pandas, Statsmodels, and Matplotlib.

Program 2

Implementation and Manipulation of Arrays Using NumPy

Problem Description: The objective of this program is to implement and manipulate arrays using the NumPy library in Python. This includes creating one-dimensional and two-dimensional arrays, accessing elements, performing mathematical operations, reshaping arrays, and applying basic array manipulation techniques commonly used in scientific computing and data analysis.

Method: The method involves importing the NumPy library and using array creation functions such as `array()`, `zeros()`, `ones()`, and `arange()`. Indexing, slicing, reshaping, and arithmetic operations are performed on both one-dimensional and two-dimensional arrays using built-in NumPy functions to demonstrate efficient handling of multidimensional data.

- Create 1D arrays using `array()` and `arange()` functions.
- Create 2D arrays using `array()`, `zeros()`, and `ones()` functions.
- Display the created arrays.
- Access specific elements from the arrays (for example, the first element of a 1D array and an element from a specific row and column in a 2D array).
- Perform array slicing to extract a row and a column from a 2D array.
- Perform mathematical operations such as addition and multiplication on arrays.
- Reshape a 1D array into a 2D array.
- Perform basic array manipulation operations such as sum, maximum value, and mean.
- Compute the row-wise and column-wise sum and mean of the 2D array.

Program Code:

```
import numpy as np
```

```
# Creating 1D arrays
```

```
arr1 = np.array([10, 20, 30, 40, 50])
```

```
arr2 = np.arange(1, 10)
```

```
# Creating 2D arrays
```

```
arr_2d = np.array([[1, 2, 3],  
                  [4, 5, 6]])
```

```
zeros_2d = np.zeros((2, 3))
```

```
ones_2d = np.ones((3, 2))
```

```
print("1D Array using array():", arr1)
```

```
print("1D Array using arange():", arr2)
```

```
print("\n2D Array using array():\n", arr_2d)
```

```
print("\n2D Array using zeros():\n", zeros_2d)
```

```
print("\n2D Array using ones():\n", ones_2d)
```

```
# Accessing elements
```

```
print("\nAccessing elements:")
```

```
print("First element of arr1:", arr1[0])
```

```
print("Element at row 2, column 3 of arr_2d:", arr_2d[1][2])
```

```
# Slicing
```

```
print("\nSlicing:")

print("First row of 2D array:", arr_2d[0])

print("Second column of 2D array:", arr_2d[:, 1])


# Mathematical operations

print("\nMathematical operations:")

print("arr1 + 5:", arr1 + 5)

print("arr_2d * 2:\n", arr_2d * 2)


# Reshaping array

reshaped_arr = arr2.reshape(3, 3)

print("\nReshaped Array (3x3):\n", reshaped_arr)


# Basic array manipulation

print("\nBasic array manipulation:")

print("Sum of arr1:", np.sum(arr1))

print("Maximum value in arr_2d:", np.max(arr_2d))

print("Mean of arr1:", np.mean(arr1))


# Row-wise sum

print("Row-wise sum:", np.sum(arr_2d, axis=1))


# Column-wise sum

print("Column-wise sum:", np.sum(arr_2d, axis=0))
```

```
# Row-wise mean

print("Row-wise mean:", np.mean(arr_2d, axis=1))

# Column-wise mean

print("Column-wise mean:", np.mean(arr_2d, axis=0))
```

Activity

A warehouse inventory system records the number of items received and dispatched for two product categories over three days. The data is stored in two 2×3 NumPy arrays, where rows represent product categories and columns represent days.

Use the following matrices:

```
received = np.array([[40, 60, 50],
                     [30, 45, 55]])
```

```
dispatched = np.array([[20, 35, 40],
                       [25, 50, 30]])
```

Perform the following tasks:

1. Perform element-wise addition of the received and dispatched arrays to compute the total number of items handled by the warehouse.
2. Perform element-wise subtraction to calculate the remaining stock.
3. If any value in the addition result is greater than 100, replace it with 100 (warehouse capacity limit).
4. If any value in the subtraction result is less than 0, replace it with 0 (stock cannot be negative).
5. Display the final addition and subtraction matrices.

Write a NumPy program to implement the above operations and print the results.

```
import numpy as np
```

```
# Creating the matrices

received = np.array([[90, 60, 50],
                     [30, 45, 55]])

dispatched = np.array([[20, 35, 40],
                       [25, 50, 30]])

print("Items Received:\n", received)
print("\nItems Dispatched:\n", dispatched)

# Element-wise addition (total items handled)
addition = received + dispatched

# Element-wise subtraction (remaining stock)
subtraction = received - dispatched

# Apply condition: replace values > 100 with 100
addition[addition > 100] = 100

# Apply condition: replace values < 0 with 0
subtraction[subtraction < 0]=0

# Display final results
```

```
print("\nAddition Result (Total Items Handled):")  
  
print(addition)  
  
print("\nSubtraction Result (Remaining Stock):")  
  
print(subtraction)
```

RESULTS:

LEARNING OUTCOMES:

Program 3

Working with Pandas DataFrames for Data Analysis

Problem Description: The objective of this experiment is to work with Pandas DataFrames to perform data analysis tasks. This includes creating DataFrames, loading data, accessing and filtering records, handling missing values, and performing basic statistical analysis to extract meaningful insights from structured data.

Method: The method involves importing the Pandas library and creating DataFrames using dictionaries or by reading data from external sources such as CSV files. Various DataFrame operations such as indexing, slicing, filtering, sorting, grouping, and applying statistical functions are performed using built-in Pandas methods to analyze and manipulate the data efficiently.

```
import pandas as pd

import numpy as np

# Create DataFrame

data = {

    'Name': ['A', 'B', 'C', 'D', None],

    'Marks': [80, 90, np.nan, 70, 60],

    'Dept': ['CS', 'EC', 'CS', 'ME', 'CS']

}

df = pd.DataFrame(data)

df.to_csv("output.csv", index=False)

# View data

print("\n First 3 Rows\n", df.head(3))
```



```

print("\n Last 2 Rows\n",df.tail(2))

print("\n Shape", df.shape)

print("\n Information:")

df.info()

print("\n Row with index 1:\n",df.loc[1])           # Row with label 1

print ("\n Row index 1 & column index 0 : ",df.loc[1, 'Name'])# Specific
value

print (" \n Row 0,1,2 : \n ",df.loc[0:2])           # Includes 0,1,2

print("\n Rows with Value>70 : \n " ,df.loc[df['Marks']>70]) # Filtering

print("\n Row with index 1:\n",df.iloc[1])           # 2nd row (position)

print ("\n Row index 1 & column index 0: ", df.iloc[1, 0])   # Row 1, Column
0

print("\n Rows 0,1 : \n" ,df.iloc[0:2])           # Only rows 0 and 1

print("\n Number of null value in Columns \n",df.isnull().sum())

print("\n Column Marks count:",df['Marks'].count() )

print("\n Mode :",df['Marks'].mode() )


# Handle missing values

df['Marks'] = df['Marks'].fillna(0)

df = df.dropna(subset=['Name'])


# Add new column

df['Result'] = df['Marks'] > 75

df['Remark'] = np.where(df['Marks'] > 75, 'Good', 'Can Improve')

```

```
# Filtering

print("\n Filtered Data:")

print(df[df['Marks'] > 70])


# Sorting

print("\n Sorted Data:")

df = df.sort_values(by='Marks', ascending=False)


# Grouping

print("\n Grouping \n", df.groupby('Dept')['Marks'].mean())


# Statistics

print("\n Statistics:")

print(df.describe())


# Save file

df.to_csv("output.csv", index=False)
```

Activity

A library wants to maintain and analyze its book issue records using a CSV file and spreadsheet tools.

Create a dataset for 10 records containing the following fields:

- **IssueID**
- **BookName**
- **DaysIssued**
- **FinePerDay**

Tasks:

1. If any of the following values are missing:
 - **DaysIssued or FinePerDay, treat them as 0.**
2. Add a new column “Status”:
 - **Mark as “No Fine” if DaysIssued = 0**
 - **Otherwise, mark as “Fine Applicable”**
3. Remove:
 - **Any rows where IssueID or BookName is empty**
4. Ensure the cleaned dataset is saved properly so it can be opened in Excel.
5. Calculate the Total Fine and add a new column:

$$\text{TotalFine} = \text{DaysIssued} \times \text{FinePerDay}$$
6. Sort the records based on TotalFine in descending order.
7. Display the top 3 highest fines.

Important Notes:

- **Include some missing values in DaysIssued or FinePerDay**
- **At least 2 records must have DaysIssued = 0 (No Fine cases)**
- **Include at least one row with missing IssueID or BookName**

Program 4:

Title: Generating Basic Plots and Visualizations Using Matplotlib

Problem Description: The objective of this experiment is to generate basic plots and visualizations using the Matplotlib library in Python. This includes creating line plots,

bar charts, histograms, and scatter plots to visually represent data and understand patterns, trends, and relationships within datasets.

Method: The method involves importing the Matplotlib library and using the pyplot module to create different types of plots. Data values are defined using Python lists or NumPy arrays, and plotting functions such as plot(), bar(), hist(), and scatter() are used. Labels, titles, and legends are added to enhance the readability and interpretation of the visualizations.

```
import matplotlib.pyplot as plt
import numpy as np

# -----
# LINE PLOT
# -----
x = np.array([1, 2, 3, 4, 5])
y = np.array([10, 20, 25, 30, 40])

plt.figure(figsize=(6,4))
plt.plot(x, y, label="Line Data")
plt.title("Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.legend(loc="upper left")
plt.show()

# -----
# BAR CHART (Color & Width)
# -----
categories = ['A', 'B', 'C', 'D']
values = [15, 25, 35, 20]

plt.figure(figsize=(5,5), dpi=120)

# Custom color and width added here
plt.bar(categories, values, color='orange', width=0.5, label="Bar
```

```

Values")

plt.title("Bar Chart (Custom Color & Width)")
plt.xlabel("Categories")
plt.ylabel("Values")
plt.legend(loc="upper right")
plt.show()

# -----
# HISTOGRAM
# -----

data = [10, 20, 20, 30, 30, 30, 40, 40, 50]

plt.figure(dpi=150)
plt.hist(data, bins=5, label="Histogram Data")
plt.title("Histogram")
plt.xlabel("Value Range")
plt.ylabel("Frequency")
plt.legend(loc="lower left", fontsize=10)
plt.show()

# -----
# SCATTER PLOT
# -----

x_scatter = [1, 2, 3, 4, 5]
y_scatter = [5, 15, 10, 25, 20]

plt.figure(figsize=(7,3))
plt.scatter(x_scatter, y_scatter, label="Scatter Points")
plt.title("Scatter Plot")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.legend()
plt.show()

```

Program 6:

Title: Case Study on Reading and Exploring Data from Text, CSV, and Excel Files for Descriptive Analytics

Problem Description: The objective of this case study is to read, explore, and analyze data from different file formats such as text files, CSV files, and Excel spreadsheets for descriptive analytics. The study focuses on understanding data structure, summarizing key characteristics, and extracting basic insights using statistical and exploratory techniques.

Method: The method involves using Python data analysis libraries such as Pandas to read data from text (read_table() or read_csv()), CSV (read_csv()), and Excel (read_excel()) files. After loading the data into DataFrames, exploratory operations such as viewing data, checking data types, handling missing values, and computing descriptive statistics are performed using built-in Pandas functions to gain meaningful insights from the datasets.

Theory Reference: Module 2

```
import pandas as pd

import numpy as np


# Read CSV with Year as index

df = pd.read_csv("file path", index_col="Year")

# Drop rows where Title is missing

print("\nMissing Values:\n", df.isnull().sum())

df = df.dropna(subset=["Title"])

print("\nMissing Values:\n", df.isnull().sum())


# Sort based on Rating

df = df.sort_values(by="Rating", ascending=False)
```

```

# Create Remark column using np.where

df["Remark"] = np.where(df["Votes"] < 50000, "Poor",

                        np.where(df["Votes"] <= 100000, "Average", "Excellent"))

# Display result

print(df.head())

# Save updated file

df.to_csv("file path")

```

Program 5:

Title: Handling Missing Data Using Single and Multiple Imputation Techniques

Problem Description: The objective of this experiment is to handle missing data in datasets using single and multiple imputation techniques. Missing data can affect the accuracy and reliability of data analysis, so this task focuses on identifying missing values and applying appropriate imputation methods to replace them for improved data quality and analysis.

Method: The method involves detecting missing values using data analysis libraries such as Pandas. Single imputation techniques such as mean, median, or mode replacement are applied to fill missing values. Multiple imputation techniques involve creating multiple plausible values for missing data using statistical models and combining the results to account for uncertainty. These techniques help in maintaining data integrity and improving analytical outcomes.

Theory Reference: Module 2

```

import pandas as pd
import numpy as np
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

```

```

# 1. Create a sample dataset with missing values
data = {
    'Experience': [1, 2, 3, np.nan, 5, 6, np.nan, 8],
    'Salary': [45000, 50000, 54000, 60000, np.nan, 70000, 75000, np.nan],
    'Type': ['Remote', 'Office', 'Office', np.nan, 'Remote', 'Remote',
'Office', 'Office']
}
df = pd.DataFrame(data)

# 2. DETECT: Check for missing values
print("Missing values per column:\n", df.isnull().sum(), "\n")

# 3. SIMPLE IMPUTATION: Handle Categorical data with Mode
df['Type'] = df['Type'].fillna(df['Type'].mode()[0])

# 4. MULTIPLE IMPUTATION (MICE): Handle Numerical data
# IterativeImputer is used to estimate Experience and Salary based on each
other
it_imputer = IterativeImputer(max_iter=10, random_state=0)
numeric_cols = ['Experience', 'Salary']
df[numeric_cols] = it_imputer.fit_transform(df[numeric_cols])

# 5. RESULT: Clean dataset
print("Cleaned Dataframe:")
print(df)

```

Program 7:

Title: Performing Statistical Analysis Using Frequency Distributions and Measures of Central Tendency and Variability

Problem Description: The objective of this experiment is to perform statistical

analysis on a given dataset by computing frequency distributions, measures of central tendency, and measures of variability. These statistical measures help in summarizing, understanding, and interpreting the underlying characteristics and spread of the data.

Method: The method involves loading the dataset using data analysis tools such as Pandas and NumPy. Frequency distributions are generated to understand the occurrence of values within the dataset. Measures of central tendency such as mean, median, and mode are calculated, along with measures of variability including range, variance, and standard deviation, using built-in statistical functions to analyze and summarize the data effectively.

Theory Reference: Module 3

1. **Import Pandas and NumPy libraries.**
2. **Create a dataset containing numerical values.**
3. **Convert the dataset into a Pandas Series.**
4. **Generate frequency distribution using value_counts().**
5. **Calculate measures of central tendency: mean, median, and mode.**
6. **Calculate measures of variability: range, variance, and standard deviation.**
7. **Display the computed statistical results.**

Program 8:

Title: Building and Validating Linear Regression Models Using Python Problem

Description: The objective of this experiment is to build and validate linear regression models using Python to understand the relationship between independent and dependent variables. The task focuses on model creation, parameter estimation, and

performance evaluation using statistical and machine learning techniques. **Method:** The method involves preparing the dataset using Pandas and NumPy, followed by splitting the data into training and testing sets. A linear regression model is built using libraries such as scikit-learn or Statsmodels. Model validation is performed by evaluating metrics such as R-squared, Mean Squared Error (MSE), and Root Mean Squared Error (RMSE) to assess the accuracy and reliability of the model. **Theory Reference:** Module 4

Write a Python program to build and validate a simple linear regression model by considering the relationship between study hours and student marks. Split the dataset into training and testing sets, determine the regression equation, and evaluate the model using R-squared, Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). Also plot the regression line.

```
import numpy as np
import matplotlib.pyplot as plt

from scipy.stats import linregress
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# Dataset: Study Hours vs Marks
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
              11, 12, 13, 14, 15, 16, 17, 18, 19, 20])

y = np.array([35, 38, 42, 45, 48, 50, 54, 57, 60, 63,
              66, 68, 71, 74, 77, 80, 82, 85, 88, 90])

# Split data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

# Build regression model using training data
result = linregress(x_train, y_train)

slope = result.slope
```

```

intercept = result.intercept

y_pred = slope * x_test + intercept

# Evaluation metrics
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

# R-squared value
r_squared = result.rvalue ** 2

# Print results
print("Slope:", slope)
print("Intercept:", intercept)
print("MSE:", mse)
print("RMSE:", rmse)
print("R-squared:", r_squared)

# Regression line for complete dataset
y_line = slope * x + intercept

# Plot
plt.scatter(x, y, color='blue', label='Actual Data')
plt.plot(x, y_line, color='red', label='Regression Line')

plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Linear Regression using linregress()")
plt.legend()

plt.show()

```

Program 9

Title: Evaluating Analysis of Variance (ANOVA) Using One-Way ANOVA in Python

Problem Description: The objective of this experiment is to evaluate the concept of

Analysis of Variance (ANOVA) to determine whether there are statistically significant differences between the means of multiple groups. The experiment focuses on applying one-way ANOVA on sample datasets and interpreting the results to support statistical decision-making.

Method: The method involves preparing a sample dataset with one independent categorical variable and one dependent numerical variable. One-way ANOVA is performed using statistical libraries such as SciPy or Statsmodels in Python. The F-statistic and p-value obtained from the analysis are examined to determine whether the null hypothesis should be accepted or rejected, thereby identifying significant differences among group means.

Problem Statement

The Iris dataset contains measurements of sepal length, sepal width, petal length, and petal width for three species of iris flowers: Setosa, Versicolor, and Virginica.

Using Python, perform a One-Way ANOVA to determine whether there is a statistically significant difference in the mean sepal length among the three iris species. Calculate the F-statistic and p-value, and interpret the results by stating whether the null hypothesis should be accepted or rejected at a 5% significance level.

Hypotheses:

- H_0 : The mean sepal length is the same for all three iris species.
- H_1 : At least one iris species has a different mean sepal length.

Tasks:

1. Load the Iris dataset.
2. Extract sepal length values for each species.
3. Perform One-Way ANOVA using Python.
4. Display the F-statistic and p-value.
5. State the decision and conclusion based on the p-value.

Program 10

Title: Case Study on Time Series Data Analysis and Forecasting Using Python

Problem Description: The objective of this case study is to analyze time series data and perform forecasting using Python. Time series analysis focuses on understanding patterns such as trend, seasonality, and cyclic behavior in time-dependent data, and forecasting aims to predict future values based on historical observations.

Method: The method involves loading and preprocessing time series data using Pandas, including date-time indexing and resampling. Exploratory analysis is carried out to identify trends and seasonal patterns. Forecasting models such as moving averages, ARIMA, or other statistical or machine learning-based 41 Version: 2.0 approaches are implemented using libraries like Statsmodels and scikit-learn. Model performance is evaluated using appropriate error metrics to assess forecasting accuracy.

Theory Reference: Module 4

Problem Statement

Using the monthly coal consumption dataset for electricity generation, perform time series analysis and forecasting in Python. Load and preprocess the data by applying date-time indexing, visualize the data to examine trends and patterns, split the dataset into training and testing periods, build an ARIMA forecasting model, generate forecasts for future coal consumption, evaluate the model using Mean Absolute Error (MAE), and compare the actual and forecasted values graphically.

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_absolute_error

# Load dataset
df = pd.read_excel("/content/Coal consumption.xlsx")

# Convert date column to datetime
df['Month'] = pd.to_datetime(df['Month'])
```

```
# Set Date as index
```

```
df.set_index('Month', inplace=True)
```

```
# Plot original time series
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(df['Value'])
```

```
plt.title('Monthly Coal Consumption for Electricity Generation')
```

```
plt.xlabel('Year')
```

```
plt.ylabel('Coal Consumption')
```

```
plt.show()
```

```
# Split data into training and testing sets
```

```
train = df['Value'][:-12]
```

```
test = df['Value'][-12:]
```

```
# Build ARIMA model
```

```
p = 12
```

```
d = 2
```

```
q = 0
```

```
model = ARIMA(train, order=(p, d, q))
```

```
results = model.fit()
```

```
# Forecast next 12 months
```

```
forecast = results.forecast(steps=12)
```

```
# Calculate MAE
```

```
mae = mean_absolute_error(test, forecast)
```

```
print("Mean Absolute Error (MAE):", mae)
```

```
# Plot actual and forecasted values
plt.figure(figsize=(10, 6))

plt.plot(train.index, train, label='Training Data')
plt.plot(test.index, test, label='Actual Data')
plt.plot(test.index, forecast, color='red', linestyle='dashed',
         label='Forecasted Values')

plt.xlabel('Year')
plt.ylabel('Coal Consumption')
plt.title('Coal Consumption Forecasting using ARIMA')
plt.legend()

plt.show()
```

Operation	Syntax	Example	Explanation
Import Pandas	import pandas as pd	import pandas as pd	Load Pandas library
Create DataFrame	pd.DataFrame(data)	df = pd.DataFrame(data)	Create table from dictionary
Read CSV	pd.read_csv("file.csv")	df = pd.read_csv("data.csv")	Load CSV file
View first rows	df.head(n)	df.head(3)	First n rows
View last rows	df.tail(n)	df.tail(2)	Last n rows
Shape of data	df.shape	df.shape	Returns (No. of rows, No.of columns)
Indexing	df.index = [new_labels]	df.index = ['a', 'b', 'c', 'd','e']	assigns new labels (names) to the rows of the DataFrame.
Column names	df.columns	df.columns	List column names
Data types	df.dtypes	df.dtypes	Show data types
Info	df.info()	df.info()	Dataset summary
Check nulls	df.isnull()	df.isnull()	True/False for nulls
Count nulls	df.isnull().sum()	df.isnull().sum()	Count missing values
Fill missing values	df.fillna(value)	df.fillna(0)	Replace null with 0
Fill missing values with mean	df['column'].fillna(df['column'].mean(), inplace=True)	df['Marks'].fillna(df['Marks'].mean(), inplace=True)	Replace null with mean

Drop missing rows	df.dropna()	df.dropna()	Remove null rows
Drop specific nulls	df.dropna(subset=['col'])	df.dropna(subset=['Name'])	Remove rows where Name is null
Select column	df['column']	df['Marks']	Access Marks column
Select multiple columns	df[['col1','col2']]	df[['Name','Marks']]	Access multiple columns
Row selection	df.loc[index]	df.loc[1]	Row with label 1
Position-based row	df.iloc[index]	df.iloc[0]	First row
Slicing	df[start:end]	df[0:2]	First 2 rows
Filtering	df[df['col'] > value]	df[df['Marks'] > 70]	Condition-based selection
Add column	df['new_col'] = value	df['Result'] = df['Marks']>75	Add new column
Sorting	df.sort_values(by='col')	df.sort_values(by='Marks')	Sort ascending
Sorting (desc)	df.sort_values(by='col', cending=False)	df.sort_values(by='Marks', ascending=False)	Sort descending
Grouping	df.groupby('col')	df.groupby('Dept')	Group data
Aggregation	df.groupby('col').mean()	df.groupby('Dept')['Marks'].mean()	Mean of groups
Count	df['col'].count()	df['Marks'].count()	Count values
Mean	df['col'].mean()	df['Marks'].mean()	Average
Max	df['col'].max()	df['Marks'].max()	Highest value
Min	df['col'].min()	df['Marks'].min()	Lowest value

Mode	<code>df['col'].mode()</code>	<code>df['Marks'].mode()</code>	Value which occurred the maost
Describe	<code>df.describe()</code>	<code>df.describe()</code>	Summary stats
Save to CSV	<code>df.to_csv("file.csv", index=False)</code>	<code>df.to_csv("output.csv", index=False)</code>	Save file
Conditional column (NumPy)	<code>np.where(condition, value_if_true, value_if_false)</code>	<code>df['Remark'] = np.where(df['Marks'] > 75, 'Good', 'Can Improve')</code>	Creates a column based on a condition (if-else logic)